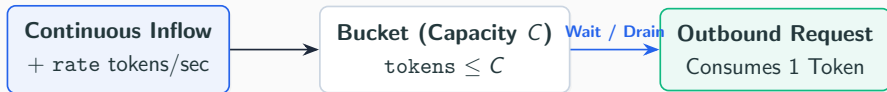


Building a Reproducible Scholarly Search Toolkit

Episode 8: Token Bucket Rate Limiting

Nexus Scholar Suite

The Token Bucket Algorithm



$$\text{tokens}(t) = \min \left(\text{capacity}, \text{tokens}(t_{last}) + (t - t_{last}) \times \text{rate} \right)$$

RateLimiter Implementation

```
class RateLimiter:
    def __init__(self, rate: float):
        self.rate = rate
        self.capacity = max(1.0, rate)
        self.tokens = self.capacity
        self.last_update = time.time()

    def wait(self) -> None:
        if self.rate <= 0: return
        while True:
            now = time.time()
            elapsed = now - self.last_update
            self.tokens = min(self.capacity, self.tokens + elapsed * self.rate)
            self.last_update = now
            if self.tokens >= 1.0:
                self.tokens -= 1.0; return
            time.sleep(max(0.01, (1.0 - self.tokens) / self.rate))
```